HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

GRADUATION RESEARCH 1

MOSAIC: A Unified Lakehouse Platform for Multi-Source CDC and Hybrid Top-*k* Analytics

Student: Chu Tuan Linh
linh.ct235963@sis.hust.edu.vn

Program: Global ICT

Supervisor: PhD. Vu Tuyet Trinh
Department: Computer Science

HANOI, 06/2026

ABSTRACT

Modern organizations increasingly depend on heterogeneous data sources – relational databases, document stores, and streaming logs – yet most open-source data infrastructures either lack a benchmark-ready reference architecture for multi-source change data capture (CDC), or fail to govern structured analytics and vector-based semantic search under one unified access-control and lineage model; furthermore, no existing approach adequately fuses exact SQL top-k results with approximate nearest-neighbor (ANN) results when the two result sets carry different degrees of data freshness. This report addresses these three gaps through MOSAIC, an open-source, self-hosted unified data platform positioned as an alternative to managed lakehouse services such as Databricks or Snowflake for organizations that require on-premise deployment. The approach taken combines a layered lakehouse architecture – Debezium-based CDC from MySQL and PostgreSQL into Apache Kafka, Apache Iceberg storage on MinIO organized in a Bronze/Silver/Gold medallion model, Apache Trino for federated SQL, pgvector for semantic retrieval, and Apache Ranger together with DataHub for governance and lineage – together with a quantitative comparison of MySQL binlog and PostgreSQL write-ahead-log (WAL) based CDC mechanisms. This report presents the resulting reference architecture, an explicit scope-decision record that separates the confirmed minimum viable core from components deferred to later phases, a clear separation of the two distinct top-k optimization tracks (SQL Top-N pushdown and ANN retrieval) that prior proposals had conflated, and a formal problem statement for freshness-aware hybrid top-k fusion proposed as a candidate research direction for Graduation Research 2 and the Thesis. The concrete deliverables of this phase are a data model for an e-commerce domain, a deployable Docker Compose stack, and Debezium connector configurations with built-in PII masking, which together provide a starting point for the subsequent implementation phase.

Student
Chu Tuan Linh

Contents

ABSTRACT **i**

LIST OF ABBREVIATIONS **v**

1 INTRODUCTION **1**

1.1 Motivation 1

1.2 Objectives and Scope of Graduation Research 1 2

1.3 Tentative Solution 3

1.4 Report Organization 3

2 PROBLEM ANALYSIS AND REQUIREMENTS **5**

2.1 Status Survey 5

2.2 Functional Requirements Overview 6

2.3 Non-functional Requirements 7

3 THEORETICAL BACKGROUND AND TECHNOLOGIES **8**

3.1 Change Data Capture and Log-based Replication 8

3.2 The Lakehouse Architecture and Apache Iceberg 8

3.3 Distributed Query Federation 9

3.4 Approximate Nearest-Neighbor Search and HNSW 10

3.5 Rank Fusion and Approximate Query Processing 10

3.6 Data Governance Technologies 11

4 SYSTEM DESIGN **13**

4.1 Software Architecture Selection 13

4.2 Overall Reference Architecture 13

4.3 Storage and Data Model Design 16

4.4 Pipeline Design and the CDC Comparison 17

4.5 Query and Performance Design 20

4.6 Governance and Security Design 21

5 SOLUTION AND CONTRIBUTION **23**
5.1 Contribution 1: A Quantitative CDC Comparison Methodology 23
5.2 Contribution 2: A Two-Track Taxonomy for Top-*k* Optimization 24
5.3 Contribution 3 (Proposed): Freshness-aware Hybrid Top-*k* Fusion . . . 24
5.3.1 Problem Statement 24
5.3.2 Research Question 25
5.3.3 Proposed Framework 25
5.3.4 Research Trajectory and Why the Problem Appears Novel . . . 26

6 CONCLUSION AND FUTURE WORK **27**
6.1 Conclusion 27
6.2 Future Work 27

REFERENCES **29**

A DATA MODEL AND ENTITY–RELATIONSHIP SCHEMA **31**
A.1 MySQL 8.0 – Transactional Source 31
A.2 PostgreSQL 15 – Master Data and Vector Store 33

B DOCKER COMPOSE STACK **35**

C DEBEZIUM CONNECTOR CONFIGURATIONS **40**
C.1 MySQL Connector (Binlog and SMT Masking) 40
C.2 PostgreSQL Connector (WAL/pgoutput and SMT Masking) 41

List of Figures

4.1 Overall reference architecture of the MOSAIC platform. 14
4.2 Entity–relationship diagram of the e-commerce reference domain. . . . 17

List of Tables

2.1 Comparison of existing data-platform categories against the position targeted by MOSAIC. 6

4.1 Rationale for each major architectural component. 15
4.2 Data sources composing the e-commerce reference domain. 16
4.3 Technical comparison of MySQL and PostgreSQL CDC mechanisms. . 18
4.4 Key performance indicators for the CDC comparison experiment. . . . 19
4.5 Two complementary masking layers. 21
4.6 Ranger role hierarchy. 22

5.1 Proposed research trajectory for the freshness-aware fusion direction. . 26

LIST OF ABBREVIATIONS

Abbreviation	Full term
ANN	Approximate Nearest Neighbor
AQP	Approximate Query Processing
CDC	Change Data Capture
DATN	"Do an tot nghiep" – the Vietnamese term for Graduation Thesis
ER	Entity–Relationship
ETL	Extract, Transform, Load
GR	Graduation Research
HNSW	Hierarchical Navigable Small World (graph)
KPI	Key Performance Indicator
LSN	Log Sequence Number
PII	Personally Identifiable Information
RBAC	Role-Based Access Control
RRF	Reciprocal Rank Fusion
SLA	Service Level Agreement
SMT	Single Message Transform
WAL	Write-Ahead Log

CHAPTER 1

INTRODUCTION

1.1 Motivation

Modern organizations, from early-stage startups to large regulated enterprises, increasingly rely on data to drive decision-making, yet the underlying data infrastructure of most organizations remains fragmented in practice. Transactional data is typically held in relational database management systems such as MySQL or PostgreSQL, which enforce strict schemas and sustain high-frequency writes; catalog and semi-structured data is commonly stored in document stores with flexible schemas; real-time events originate from application logs, application programming interface (API) calls, and clickstreams at high volume; and, increasingly, semantic use cases require vector embeddings to support similarity search for artificial intelligence and machine learning workloads. Each of these systems exposes different semantics, protocols, and performance characteristics, so organizations that need a coherent, governed view across all of them currently have to assemble ad-hoc integrations rather than adopt a single reference platform.

This fragmentation carries a direct cost. Without a benchmark-ready reference architecture for multi-source change data capture, engineering teams routinely spend several months on integration work without any established baseline to validate their design against. Without a unified governance model spanning both structured analytics and vector indexes, personally identifiable information can leak through embeddings that are not subject to the same masking and access-control policies as relational tables, and end-to-end data lineage becomes impossible to reconstruct. Additionally, when a platform serves results from both an exact structured-query pipeline and an approximate semantic-search pipeline, naively fusing their rankings ignores the fact that the two pipelines typically update at very different rates, which can cause the combined result to misrepresent the true, current state of the underlying data. These three problems – reference-architecture scarcity, governance fragmentation, and freshness-unaware result fusion – motivate the present research.

Several factors specific to the Vietnamese and wider ASEAN context reinforce the practical relevance of a self-hosted approach. Organizations in regulated sectors

such as finance, healthcare, and government are frequently subject to data-sovereignty requirements that preclude sending data to a foreign-hosted managed cloud service. Managed lakehouse platforms such as Databricks and Snowflake carry licensing costs that are difficult to justify for many small and medium enterprises. At the same time, demand for artificial-intelligence features such as semantic search, recommendation, and fraud detection continues to grow, and all of these features require a unified data foundation rather than a collection of disconnected systems.

1.2 Objectives and Scope of Graduation Research 1

A review of the current landscape of data-platform technologies indicates three categories of existing solutions, none of which fully addresses the problems described in Section 1.1. Managed cloud platforms, including Databricks, Snowflake, and BigQuery, are feature-complete but impose considerable licensing cost, create vendor lock-in, and typically require data to leave the organization's own infrastructure; furthermore, their vector-search capabilities have generally been added as proprietary bolt-on features rather than being integrated into the same governance model as structured tables. Legacy Hadoop-ecosystem platforms, such as Cloudera and the Hortonworks Data Platform, are self-hosted but rest on an aging HDFS-based architecture that lacks native real-time CDC and vector-search support and is comparatively difficult to operate. Fully do-it-yourself open-source stacks are flexible but offer no reference architecture, so every adopting team must independently integrate each component without a documented baseline for comparison.

On the basis of this landscape review, the objective of Graduation Research 1 (GR1) is defined as the production of a feasibility and design study for MOSAIC, an open-source, self-hosted unified data platform intended for data-engineering teams that need capabilities comparable to Databricks or Snowflake but cannot, or choose not to, depend on managed cloud. Three specific gaps are targeted: (i) the absence of a benchmark-ready reference architecture that unifies real-time CDC from heterogeneous relational sources with lakehouse storage; (ii) the lack of an open-source platform that governs structured analytics and vector search under one role-based access-control and lineage model; and (iii) the lack of an optimized approach to hybrid top-k fusion when combining exact SQL results with approximate vector results of differing freshness.

As this is a feasibility and design phase rather than a full implementation phase, GR1 is scoped as a study that produces (i) a justified reference architecture and a set

of architectural decisions, (ii) a small number of concrete engineering deliverables – a data model, a deployable stack definition, and connector configurations – and (iii) an explicit record of which components from the original, broader proposal are retained in the minimum viable core and which are deliberately deferred to Graduation Research 2 (GR2) or the Thesis. This scope decision, together with the rationale behind it, is documented in full in Section **??**.

1.3 Tentative Solution

The approach adopted in this research follows the lakehouse architectural pattern, which unifies the data lake and the data warehouse on open, direct-access storage formats while retaining warehouse-grade query performance [1]. Concretely, the tentative solution combines log-based change data capture from MySQL and PostgreSQL through Debezium and Apache Kafka, storage on Apache Iceberg over MinIO organized into a Bronze/Silver/Gold medallion model, federated SQL querying through Apache Trino, semantic retrieval through the pgvector extension of PostgreSQL, and governance enforced jointly by Apache Ranger and DataHub. This combination is chosen because it allows one system – PostgreSQL – to serve simultaneously as a CDC source and as the vector store, which yields a natural dual-role architecture, and because each remaining component is selected specifically for its support of open standards and self-hosted deployment.

The principal contribution of this research phase is threefold: a quantitative, reproducible comparison of MySQL binlog-based and PostgreSQL WAL-based CDC, an explicit separation of two previously conflated research tracks under the label "top-k optimization" – SQL Top-N pushdown in a federated query engine, and approximate nearest-neighbor retrieval in a vector store – and a formal problem statement for freshness-aware hybrid top-k fusion, proposed as a candidate research direction for later phases rather than claimed as completed work in GR1.

1.4 Report Organization

The remainder of this report is organized as follows. Chapter 2 surveys the existing landscape of data-platform technologies in more detail and derives the functional and non-functional requirements that MOSAIC must satisfy. Chapter 3 introduces the theoretical background and the specific technologies adopted, connecting each choice back to a requirement identified in Chapter 2. Chapter 4 presents the system design, including the reference architecture, the data model, the pipeline and query-

performance design, the governance design, and the explicit scope-decision record that separates the confirmed minimum viable core from deferred components. Chapter 5 presents the solution and the contributions considered most significant in this research phase, namely the CDC comparison methodology, the two-track top-k taxonomy, and the proposed freshness-aware fusion problem. Chapter 6 concludes the report and outlines the planned direction for Graduation Research 2 and the Thesis.

CHAPTER 2

PROBLEM ANALYSIS AND REQUIREMENTS

Chapter 1 introduced the fragmentation problem that motivates this research and set out the three gaps that MOSAIC is intended to address. This chapter surveys the existing landscape of data-platform technologies in greater detail, positions MOSAIC within that landscape, and derives the functional and non-functional requirements that guide the architectural decisions presented in Chapter 4. Because MOSAIC is an infrastructure platform rather than an end-user application, its requirements are expressed in terms of the data flows and governance guarantees it must support, rather than in terms of end-user use cases.

2.1 Status Survey

Table 2.1 summarizes the comparison, introduced in Section 1.2, between the three categories of existing solutions and the position targeted by MOSAIC. Managed cloud platforms provide the broadest feature set but require organizations to relinquish control over where data is physically stored and to accept a recurring licensing cost; their recently added vector-search features are typically proprietary and are not exposed through the same access-control layer as structured tables. Hadoop-ecosystem platforms are self-hosted but rest on an HDFS-based storage layer that predates modern open table formats, so they lack native real-time CDC ingestion and offer no vector-search capability at all. Fully do-it-yourself stacks assembled from individual open-source components are maximally flexible, but every adopting organization has to independently resolve integration questions – such as how to keep Trino table statistics fresh after Spark writes, or how to prevent PostgreSQL replication-slot accumulation during a CDC outage – without a documented reference to consult.

Table 2.1: Comparison of existing data-platform categories against the position targeted by MOSAIC.

Category	**Strengths**	**Limitations**
Managed cloud (Databricks, Snowflake, BigQuery)	Feature-complete; low operational burden	High cost; vendor lock-in; data leaves the organization; vector search is a proprietary bolt-on
Legacy Hadoop ecosystem (Cloudera, HDP)	Self-hosted; mature batch processing	Aging HDFS-based architecture; no native real-time CDC; no vector search
Do-it-yourself open-source stack	Maximum flexibility; no licensing cost	No reference architecture; integration burden falls entirely on the adopting team
MOSAIC (this work)	**Self-hosted; open standards; documented reference architecture and benchmark design**	**Requires in-house operational capability (addressed by the deployable stack in Appendix B)**

The comparison in Table 2.1 indicates that no existing category simultaneously offers self-hosted deployment, a documented reference architecture, and unified governance across structured and vector data. The functional and non-functional requirements derived in the remainder of this chapter are formulated to close precisely this combination of gaps.

2.2 Functional Requirements Overview

The functional requirements of MOSAIC are organized by architectural layer, consistent with the layered lakehouse pattern adopted in Section 1.3. At the ingestion layer, the platform must capture row-level insert, update, and delete events from both a MySQL and a PostgreSQL source with no missed events across a connector restart, and it must mask personally identifiable information before any event reaches persistent storage. At the storage layer, the platform must organize ingested data into progressively cleaner tiers – raw, cleaned, and aggregated – on an open table format that supports atomic commits and historical snapshots, and it must additionally store vector embeddings for a subset of the domain data to support semantic retrieval. At

the serving layer, the platform must expose a single SQL interface capable of querying the lakehouse tables and, where useful for live lookups, federating queries against the operational databases directly. At the governance layer, the platform must enforce role-based, column-level access control and masking at query time, and it must automatically capture lineage linking each curated table back to its originating source system.

2.3 Non-functional Requirements

Beyond the functional requirements above, four non-functional requirements shape the architecture. First, the platform must be deployable entirely on-premise, with no mandatory dependency on any external managed service, in order to satisfy the data-sovereignty constraints described in Section 1.1. Second, the CDC pipeline must recover correctly after a connector restart without losing or duplicating events, since this restart-safety property is a precondition for treating the platform as a reliable source of truth. Third, the platform must guard against the PostgreSQL write-ahead-log accumulation risk inherent to logical replication, since an unmonitored replication slot can otherwise exhaust disk space during a prolonged consumer outage. Fourth, the platform's design must remain extensible enough to accommodate the components deferred from GR1 – including an additional document-store source, a purpose-built vector engine, and the freshness-aware fusion mechanism – without requiring a redesign of the core data flow; this extensibility requirement is what allows the scope decisions recorded in Section **??** to defer components safely rather than discard them.

This chapter has surveyed the existing data-platform landscape, positioned MOSAIC relative to it, and derived the functional and non-functional requirements that the remaining chapters address. Chapter 3 now introduces the theoretical background and specific technologies selected to satisfy these requirements.

CHAPTER 3

THEORETICAL BACKGROUND AND TECHNOLOGIES

Chapter 2 derived the functional and non-functional requirements that MOSAIC must satisfy. This chapter introduces the theoretical foundations and the specific technologies selected to meet those requirements, and for each technology it identifies which requirement from Chapter 2 it addresses as well as the alternative approaches that were considered.

3.1 Change Data Capture and Log-based Replication

Change data capture is the general technique of detecting and recording insert, update, and delete operations in a source database so that they can be propagated incrementally to a destination system, rather than through periodic full-table reloads [6]. Three mechanisms are commonly used to implement CDC. Log-based CDC reads the source database's transaction log directly and offers zero additional write overhead on the production system, atomic capture of every event type, and a guaranteed ordering of events, at the cost of requiring elevated replication privileges. Trigger-based CDC writes changes into an audit table through a database trigger, which is comparatively simple to implement but increases write overhead and complicates the source schema. Polling-based CDC periodically queries a table for rows modified since the last check, which requires no special privileges but introduces latency and, critically, cannot detect deletions that leave no trace in an `updated_at` column. Log-based CDC is adopted in this work because the zero-overhead and guaranteed-ordering properties directly satisfy the restart-safety non-functional requirement identified in Section 2.3, and because it is the mechanism implemented by Debezium, the CDC platform selected for this project [8].

3.2 The Lakehouse Architecture and Apache Iceberg

The lakehouse architecture was proposed as a unification of the data lake and the data warehouse on open, direct-access storage formats, combined with first-class

support for machine-learning workloads and warehouse-grade query performance [1]. This pattern is adopted as the theoretical foundation of MOSAIC's storage layer because it satisfies the requirement, identified in Section 2.2, for a single source of truth that supports both historical analytics and machine-learning consumption without duplicating data into separate systems.

Apache Iceberg is selected as the concrete open table format implementing this pattern. Iceberg tracks table state through metadata files, and every write creates a new metadata file that atomically replaces the previous one, which gives the table atomic-commit and consistent-read semantics; a sequence of such metadata files forms a chain of snapshots that represent the table's state at successive points in time, which is the mechanism underlying Iceberg's time-travel capability [9]. Manifest files associated with each snapshot track the individual data files together with column-level statistics such as minimum and maximum values and null counts, which a query engine uses for scan planning. Because this metadata-based approach differs fundamentally from the B-tree indexing used in online-transaction-processing databases, Iceberg's optimization mechanisms are more accurately described as a layered pruning strategy: partition pruning, using hidden partition transforms, allows an engine to skip entire partitions that cannot match a query predicate; column statistics allow an engine to skip individual files that cannot contain a qualifying row; Bloom filters, enabled per column through table properties, accelerate high-cardinality equality lookups; and sort order or Z-ordering clusters data along multiple columns simultaneously, which sharpens the effectiveness of the column-statistics layer and reduces the number of bytes an engine must scan [9, 14].

3.3 Distributed Query Federation

Apache Trino is a distributed SQL query engine whose central capability is query federation: the ability to query object storage, relational databases, and other heterogeneous sources within a single SQL statement through a common connector interface [10]. Trino is selected for the serving layer because it directly satisfies the requirement, identified in Section 2.2, for one SQL interface over both the lakehouse and live operational sources; the alternative considered was to expose each source through its own native query interface and require downstream consumers to perform cross-source joins in application code, which was rejected because it would defeat the purpose of unification. Trino's connector application-programming interface allows an individual connector to implement an `applyTopN` method, which lets Trino push

a `ORDER BY ... LIMIT` operation down into the connector so that only the requested number of rows is returned rather than the full result set [10]; this pushdown mechanism is the practical basis of the SQL Top-N optimization track studied in Section 5.2.

3.4 Approximate Nearest-Neighbor Search and HNSW

Approximate nearest-neighbor search addresses the problem of retrieving, for a query vector and an integer k, the k vectors most similar to the query among a large collection, while accepting a controlled probability of missing the exact nearest neighbors in exchange for substantially lower latency. Hierarchical Navigable Small World (HNSW) graphs are a widely adopted approach to this problem: the index is organized as a multi-layer proximity graph in which higher layers contain progressively sparser subsets of nodes, so that a search begins with coarse navigation at the top layer and progressively refines toward the query vector as it descends toward the bottom layer, typically achieving a recall above 95 percent at millisecond latency on collections of several million vectors [2]. Three parameters govern the accuracy-latency trade-off of an HNSW index: (i) M, the number of graph connections maintained per node, where a larger value improves accuracy at the cost of memory; (ii) ef_construction, the search-expansion factor used while building the index, where a larger value produces a higher-quality index at the cost of longer build time; and (iii) ef_search, the search-expansion factor used at query time, where a larger value improves recall at the cost of query latency. HNSW is adopted in this work through the pgvector extension for PostgreSQL, which implements an HNSW index alongside exact and other approximate nearest-neighbor methods while keeping vectors co-located with the rest of the relational data [13]; the alternative considered was Milvus, a purpose-built vector database offering additional index families such as IVF-Flat and IVF-PQ, which is deferred to Graduation Research 2 as recorded in Section **??**.

3.5 Rank Fusion and Approximate Query Processing

When a platform must combine ranked results produced by two different retrieval mechanisms, a rank-fusion method is required. Reciprocal Rank Fusion (RRF) is the most widely used baseline for this problem; it assigns each document a score equal to the sum, over every ranked list in which the document appears, of the reciprocal of a small constant k plus the document's rank in that list [7]. RRF treats every input ranking as equally current and does not model the possibility that the rankings were

computed from data of differing freshness, a limitation that motivates the freshness-aware fusion problem formulated in Section 5.3.

A related but distinct body of theory concerns approximate query processing over very large datasets, where returning an approximate result in a fraction of a second is often preferable to waiting tens of seconds for an exact result. BlinkDB demonstrated a sampling-based approach that provides bounded error and bounded response time for interactive queries over massive data [4], and VerdictDB later generalized this idea into a middleware layer that rewrites queries to obtain approximate answers on top of an unmodified relational engine [5]. Trino implements a practical instance of this theory directly: the built-in `approx_distinct` function uses a HyperLogLog sketch to estimate cardinality with roughly one to two percent error using only a few kilobytes of memory, and `approx_percentile` uses a t-Digest sketch to estimate percentiles accurately even in the tail of a distribution [10]. These functions are noted here as a practical lever available to MOSAIC without requiring a dedicated approximate-query engine to be built, though the corresponding benchmark experiments are deferred to GR2 as recorded in Section **??**.

3.6 Data Governance Technologies

Two complementary technologies satisfy the governance requirement identified in Section 2.2. Apache Ranger is a centralized authorization framework supporting role-based, attribute-based, and tag-based access control together with centralized auditing, and its Trino plugin specifically supports catalog-, schema-, table-, and column-level authorization together with dynamic column masking and row filtering [11]. DataHub is a metadata platform that provides a data catalog, a column-level lineage graph, and native ingestion of lineage events emitted in the OpenLineage format, which allows Apache Airflow and Apache Spark to emit lineage automatically as pipelines run, without requiring changes to individual pipeline code [12]. The alternative considered for authorization was to implement masking exclusively at the ingestion layer through Kafka Connect Single Message Transforms, which is retained as a first, irreversible layer of protection but is judged insufficient on its own because it cannot express role-dependent masking policies at query time; the two mechanisms are therefore combined, as detailed in Section 4.6.

This chapter has introduced the theoretical background for change data capture, the lakehouse storage model, query federation, approximate nearest-neighbor search, rank fusion, and data governance, connecting each technology choice to a require-

ment derived in Chapter 2. Chapter 4 now presents the concrete system design that assembles these technologies into the MOSAIC reference architecture.

CHAPTER 4

SYSTEM DESIGN

Chapter 3 introduced the technologies selected to satisfy the requirements derived in Chapter 2. This chapter presents the concrete system design that assembles those technologies into the MOSAIC reference architecture: the overall software-architecture selection, the reference architecture diagram, the data model, the pipeline design, the query-performance design, the governance design, and, finally, the explicit scope-decision record that separates the confirmed minimum viable core from the components deferred to later phases.

4.1 Software Architecture Selection

MOSAIC adopts a layered lakehouse architecture, in which ingestion, processing, storage, serving, and governance are treated as first-class, independently scalable layers rather than as an undifferentiated pipeline. This selection follows directly from the lakehouse pattern introduced in Section 3.2: applying the general pattern to MOSAIC's concrete sources and requirements yields six layers, namely a source layer comprising MySQL and PostgreSQL, an ingestion layer comprising Debezium and Apache Kafka, a processing layer comprising Apache Spark Structured Streaming together with Great Expectations quality gates, a storage layer comprising Apache Iceberg on MinIO together with the pgvector store, a serving layer comprising Apache Trino, and a governance layer, cutting across all other layers, comprising Apache Ranger and DataHub. Each layer scales independently of the others, which satisfies the extensibility non-functional requirement identified in Section 2.3.

4.2 Overall Reference Architecture

Figure 4.1 presents the resulting reference architecture. Data flows from the two relational sources through Debezium connectors into Kafka topics, where Kafka Connect Single Message Transforms apply the first, irreversible layer of PII masking before any record is persisted; Spark Structured Streaming then reads these topics, normalizes the Debezium change envelope, and writes into the Iceberg medallion layers on MinIO, while Airflow orchestrates the batch transformations, quality gates, and

Iceberg maintenance jobs that operate on top of this flow. Trino serves federated SQL queries against both the Iceberg tables and the live relational sources, pgvector serves semantic-similarity queries against the embeddings generated from the Gold-layer product catalog, and Ranger together with DataHub enforce access control, masking, and lineage across every layer.

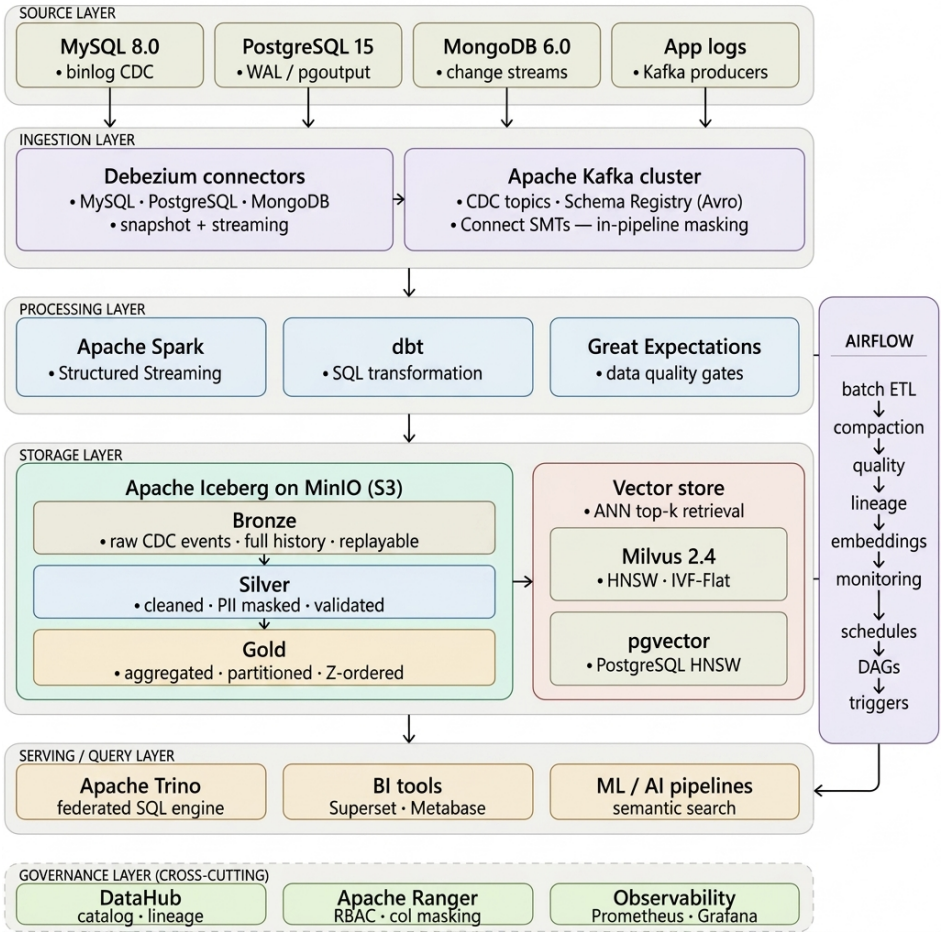


Figure 4.1: Overall reference architecture of the MOSAIC platform.

Table 4.1 summarizes the rationale behind each major component shown in Figure 4.1, connecting every choice back to the theoretical background presented in Chapter 3.

Table 4.1: Rationale for each major architectural component.

Component	**Rationale**
Apache Iceberg	Open table format with a snapshot/metadata model for atomic commits, time-travel, and hidden partitioning; supported natively by every major query engine considered [9].
Debezium	Open-source CDC covering MySQL binlog and PostgreSQL WAL within one framework, with restart safety achieved through Kafka-stored source offsets [8].
Apache Kafka	Durable streaming buffer that provides replay and independent fan-out consumers, decoupling the scaling of sources from the scaling of sinks [10].
Apache Trino	Query federation over object storage and relational databases within one SQL engine, with Top-N pushdown and dynamic filtering support [10].
MinIO	Self-hosted, S3-API-compatible object store used as the lakehouse back end.
Apache Ranger	Centralized authorization with a Trino plugin supporting column masking, row filtering, and audit logging [11].
Kafka Connect SMTs	In-pipeline masking so that raw PII never persists to any storage tier.
DataHub	Automatic catalog and lineage ingestion from Airflow and Spark through the OpenLineage format [12].
pgvector	Vector similarity search inside PostgreSQL, enabling native SQL joins to structured metadata and a dual role with the CDC source [13].

Data within the Iceberg lakehouse is organized according to the medallion pattern, reflecting increasing levels of processing and quality. The Bronze tier stores the unmodified Debezium change envelope, including the before/after row state, the operation type, the source offset or log sequence number, and the event timestamp, with no transformation applied, in order to preserve full auditability and replay capability. The Silver tier stores data that has been deduplicated, normalized, type-cast, masked, and validated by Great Expectations checkpoints. The Gold tier stores data that has been joined and aggregated according to domain business logic, with time-based partitioning and, where beneficial, Z-ordering applied; this is the tier that Trino

and downstream analytics tools primarily query. The write strategy connecting these tiers is that Spark Structured Streaming reads the Debezium topics, normalizes the change envelope, applies deduplication using the source offset as an idempotency key, appends the result to Bronze, and merges or upserts the result into Silver and Gold.

4.3 Storage and Data Model Design

The domain selected for the reference implementation is electronic commerce, because it is realistic and exercises every data type required by the platform. Table 4.2 lists the four data sources that compose this domain and the role each one plays.

Table 4.2: Data sources composing the e-commerce reference domain.

Source	**System**	**Content**
MySQL	Transactions	Orders, order items, payments
PostgreSQL	Master data	Users, products, categories, inventory
Derived (pgvector)	Vector index	Product-description embeddings (384-dimensional)

PostgreSQL plays a dual role in this design: it is simultaneously a CDC source for master data and, through the pgvector extension, the vector store for semantic search, as discussed in Section 3.4. The complete data definition language for both sources is provided in Appendix A; Figure 4.2 presents the corresponding entity–relationship diagram.

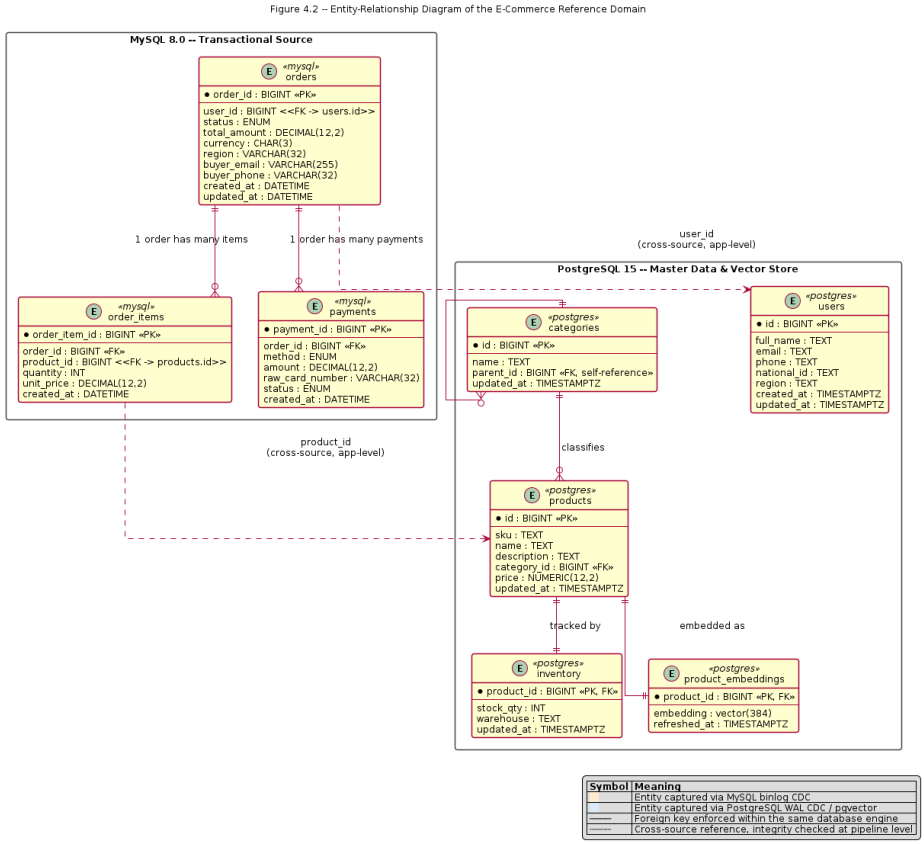


Figure 4.2: Entity–relationship diagram of the e-commerce reference domain.

As shown in Figure 4.2, referential integrity between the MySQL and PostgreSQL clusters – for example, between an order's `user_id` and the corresponding row in the PostgreSQL `users` table – cannot be enforced by either database engine individually, since the two tables reside in separate database systems; this cross-source integrity is instead maintained at the pipeline level, through the consistency checks implemented in the Great Expectations quality gates introduced in Section 4.1. For the Gold layer, orders are partitioned by `PARTITIONED BY (days(created_at), bucket(16, region))` to support both time-series and regional analytics, users are partitioned by `bucket(32, user_id)` to support user-specific lookups, and the Silver layer is partitioned by source system and date to simplify replay.

4.4 Pipeline Design and the CDC Comparison

The retained comparison between MySQL binlog-based and PostgreSQL WAL-based CDC constitutes the quantitative core of this research phase, since it is designed to yield an evidence-based recommendation rather than a purely descriptive account. Table 4.3 presents the technical comparison between the two mechanisms.

Table 4.3: Technical comparison of MySQL and PostgreSQL CDC mechanisms.

Criterion	**MySQL 8.0**	**PostgreSQL 15**
Transaction log	Binary log (binlog)	Write-ahead log (WAL)
Debezium mechanism	Reads binlog through the binlog protocol	Logical replication slot with the pgoutput plugin
Required log format	`binlog_format=ROW`, `binlog_row_image=FULL`	`wal_level=logical`
Position tracking	Binlog file name and byte position	Log sequence number, e.g. `0/1A2B3C4D`
DDL capture	Well supported through a schema-history topic	More limited under the pgoutput plugin
Restart safety	Offset stored in the Kafka Connect offsets topic	LSN stored in the replication slot
Log-accumulation risk	Low, governed by `expire_logs_days`	**High**: the replication slot retains WAL until consumed
Dual-role potential	OLTP source only	**Yes**: PostgreSQL combined with pgvector

As Table 4.3 indicates, the most significant operational risk specific to the PostgreSQL path is write-ahead-log accumulation: because a logical replication slot prevents PostgreSQL from removing WAL segments until Debezium has processed them, a prolonged Debezium outage causes WAL to accumulate on disk and can eventually exhaust available storage. This risk is mitigated in the reference implementation by monitoring `pg_replication_slots.confirmed_flush_lsn`, by bounding retained WAL through the `wal_keep_size` setting, and by emitting periodic heartbeat events that advance the confirmed-flush position even when the monitored tables receive no application writes; MySQL does not face an equivalent risk, since binlog retention is governed by the independent `expire_logs_days` policy. Both mechanisms are normalized by Debezium into one unified change-event envelope, illustrated in Listing 4.1, so that downstream consumers such as the Spark Structured Streaming job introduced in Section 4.2 do not need to know which source system produced a given event.

```
{
  "op": "c|u|d|r",
  "before": { },
  "after":  { },
```

```
  "source": {
    "db": "ecommerce", "table": "orders", "ts_ms": 1699123456789,
    "file": "mysql-bin.000001", "pos": 1234,   // MySQL specific
    "lsn": "0/1A2B3C4D", "txId": 492           // PostgreSQL specific
  }
}
```

Listing 4.1: Unified Debezium change-event envelope.

The comparison design specifies five key performance indicators, listed in Table 4.4, to be measured once the pipeline is implemented in GR2: end-to-end latency from source commit to availability in the Iceberg Bronze tier, sustained event throughput under load, the time required to catch up after a connector restart, the downstream impact of a schema-altering statement on the source table, and, specific to PostgreSQL, the WAL lag between the confirmed-flush position and the current LSN.

Table 4.4: Key performance indicators for the CDC comparison experiment.

Indicator	**Measurement method**
End-to-end latency	Time from source commit to row availability in Iceberg Bronze
Throughput	Events processed per second under sustained load
Restart recovery time	Elapsed time to catch up after stopping and restarting the connector while writes continue at the source
DDL handling	Downstream impact of an `ALTER TABLE` statement issued at the source
WAL lag (PostgreSQL only)	`confirmed_flush_lsn` compared against the current LSN

Kafka Connect Single Message Transforms apply the first, irreversible layer of masking immediately after the source connector produces a record, before that record enters any Kafka topic. The `MaskField` transform replaces sensitive field values, such as an email address or phone number, with a null value; the `ReplaceField` transform removes a sensitive field entirely, such as a raw card number; and the `TimestampConverter` transform harmonizes the differing timestamp representations produced by MySQL and PostgreSQL into a single ISO 8601 format. The concrete connector configurations implementing these transforms are provided in Appendix C.

4.5 Query and Performance Design

Trino serves as the platform's federation layer, connecting to Iceberg for primary analytics with full pushdown support, and to MySQL and PostgreSQL for live operational lookups with more limited pushdown support. Cross-source joins benefit from dynamic filtering, in which filter values computed from a small dimension table are propagated into the scan of a larger fact table to reduce the number of bytes read. Listing 4.2 illustrates a representative federated query joining the Iceberg Gold orders table with the live PostgreSQL users table.

```
SELECT o.order_id, u.full_name, o.total_amount
FROM iceberg.gold.orders o
JOIN postgresql.public.users u ON o.user_id = u.id
WHERE o.created_at >= DATE '2024-01-01' AND o.status = 'delivered'
ORDER BY o.total_amount DESC
LIMIT 100;
```

Listing 4.2: Representative federated query across Iceberg and PostgreSQL.

Because Trino computes plan-node costs from table statistics, and because Spark writes to Iceberg outside of Trino's own visibility, statistics must be refreshed explicitly through the `ANALYZE` statement after each batch write; without this refresh, the cost-based optimizer can select a sub-optimal join order or probe side. The corresponding experiment design specifies three steps: recording the query plan, scanned bytes, and wall-clock time for a baseline query executed without statistics; running `ANALYZE`; and then recording the same measurements for the identical query, so that the join-order change, CPU-time change, and scanned-bytes change can be attributed solely to the presence of statistics. It is important to acknowledge, rather than conceal, three engineering caveats that this design surfaces: federation is not a uniform performance guarantee, since pushdown support differs across connectors, with Iceberg offering the strongest support; statistics freshness requires an explicit `ANALYZE` step after every Spark write, which is an operational overhead that must be scheduled; and, as discussed in Section 4.4, the PostgreSQL replication slot can accumulate WAL if Debezium lags. These caveats are treated as a deliberate part of the scope, in that they justify why query-optimization experiments are included in the GR1 design rather than assumed away.

As introduced in Section 3.3 and Section 3.4, the term "top-k optimization" in this research separates into two independent tracks, and this separation is treated as one of the contributions presented in Chapter 5. The SQL Top-N pushdown track

addresses the problem of minimizing input/output and network transfer for a query of the form `ORDER BY ... LIMIT` k executed over federated sources, using Trino's `applyTopN` connector method described in Section 3.3; its experiment design compares the `EXPLAIN` plan, scanned bytes, and wall-clock time with and without pushdown enabled, on tables of at least ten million rows so that the difference is statistically meaningful, while isolating the pushdown setting as the only varied condition. The approximate nearest-neighbor track addresses the different problem of retrieving the k most similar vectors to a query vector with an acceptable recall and latency, using the pgvector HNSW index described in Section 3.4; its experiment design sweeps the M and ef_search parameters, plots recall at k against latency, compares the HNSW index against exact brute-force search, and reports the 50th-, 95th-, and 99th-percentile latency together with the memory overhead of the index.

4.6 Governance and Security Design

Governance in MOSAIC is enforced through two complementary layers, summarized in Table 4.5. The first layer, implemented through Kafka Connect SMTs as described in Section 4.4, applies masking at ingestion time, before any record enters Kafka, so that raw PII never persists to any storage tier; this layer is irreversible by design. The second layer, implemented through the Trino Ranger plugin introduced in Section 3.6, applies column masking dynamically at query time according to the requesting user's role, so that the same underlying Bronze-tier data can be served differently to different roles without duplicating storage.

Table 4.5: Two complementary masking layers.

Layer	**Mechanism and timing**	**Fate of PII**
Layer 1 – Kafka Connect SMT	`MaskField` / `ReplaceField` at ingestion, before the Kafka topic	Irreversible; never persisted
Layer 2 – Trino + Ranger	Column masking at query time, role-dependent	Raw value exists in Bronze; masked view is served

Table 4.6 defines the role hierarchy enforced by Ranger. A `data_admin` role has unrestricted access to every layer, including raw PII, and is intended for the platform and database-administration team only; a `data_engineer` role may read and

write the Silver and Gold tiers and may read the Bronze tier only in masked form; a `data_analyst` role has read-only access to the Gold tier with automatic column masking applied; an `ml_engineer` role may read the Gold tier and the vector store, but only in anonymized form; and a `bi_user` role is restricted to read-only access to pre-aggregated Gold views.

Table 4.6: Ranger role hierarchy.

Role	**Access**
`data_admin`	Full access to every layer; raw PII visible; restricted to the platform and database-administration team
`data_engineer`	Read/write access to Silver and Gold; read access to Bronze in masked form
`data_analyst`	Read-only access to Gold, with automatic column masking
`ml_engineer`	Read access to Gold and the vector store, in anonymized form only
`bi_user`	Read-only access to pre-aggregated Gold views

Beyond masking and role-based access, three further controls complete the governance design. In transit, Kafka connections are protected by SSL/TLS and SASL, MinIO is accessed over HTTPS, Trino enforces TLS, and the PostgreSQL connection used by Debezium requires `sslmode=require`. At rest, MinIO applies server-side encryption with keys managed by a key management service. Every Trino query is logged, including the requesting user, the query text, the tables accessed, the timestamp, and the execution time, into an immutable Iceberg audit table, which additionally supports a right-to-erasure workflow through Iceberg's row-level, merge-on-read delete mechanism, avoiding the need to rewrite an entire Parquet file to remove a single row.

CHAPTER 5

SOLUTION AND CONTRIBUTION

The preceding chapters described the requirements, the theoretical background, and the overall system design of MOSAIC. This chapter presents, in dedicated detail, the three pieces of work considered the most significant intellectual contribution of this research phase: a quantitative benchmark design for comparing MySQL and PostgreSQL as CDC sources, an explicit separation of two previously conflated top-k optimization tracks, and a formal problem statement for freshness-aware hybrid top-k fusion, proposed as a candidate direction for later research. Each contribution is presented through its motivating problem, the solution or formalization developed, and the result expected once the corresponding experiment is executed in a later phase; content already presented in full in Chapter 4 is not repeated here, and cross-references are used instead.

5.1 Contribution 1: A Quantitative CDC Comparison Methodology

Most existing tutorials and reference architectures for CDC-based pipelines assume a single, homogeneous relational source, and consequently offer no quantitative basis for choosing between MySQL and PostgreSQL when a new system is designed specifically around CDC as a first-class requirement. The solution developed in this research, summarized in Section 4.4, is a benchmark design that captures the same workload simultaneously through both mechanisms and measures the five indicators listed in Table 4.4 under identical conditions. Because both connectors normalize their output into the same change-event envelope shown in Listing 4.1, the downstream measurement code does not need to vary between the two sources, which removes a common confound from this class of comparison. The expected result of this benchmark, once executed in GR2, is a defensible, evidence-based recommendation for which mechanism to prefer under which operating conditions – for example, whether the WAL-accumulation risk identified in Table 4.3 materially affects restart-recovery time under realistic outage durations – rather than a recommendation based on documentation alone.

5.2 Contribution 2: A Two-Track Taxonomy for Top-*k* Optimization

The original proposal that preceded this report grouped SQL Top-N pushdown and approximate nearest-neighbor retrieval under a single label, "top-*k* optimization," which conflates two problems with fundamentally different error models: SQL Top-N pushdown is concerned with exactness and with minimizing the number of accesses needed to compute a provably correct answer, following the threshold-algorithm theory of Fagin, Lotem, and Naor [3], whereas approximate nearest-neighbor retrieval is concerned with a controlled recall-latency trade-off over a geometric search space, following the HNSW theory summarized in Section 3.4. Treating these two problems as one, as the original proposal did, risks designing a single experiment that answers neither question well. The solution developed in this research is to separate them explicitly into Track 1, SQL Top-N pushdown, and Track 2, approximate nearest-neighbor retrieval, with the independent experiment designs already presented in Section 4.5. This separation is itself a contribution because it corrects a conceptual error present in the original proposal, and it is expected, once both experiments are executed, to yield two independent sets of results – a scanned-bytes and wall-clock-time comparison for Track 1, and a recall-versus-latency curve for Track 2 – that can each be evaluated against the correct theoretical baseline for its own problem, rather than against a single, ill-defined combined baseline.

5.3 Contribution 3 (Proposed): Freshness-aware Hybrid Top-*k* Fusion

This section documents a candidate research direction under consideration for Graduation Research 2 and the Thesis. It is not claimed as completed work within GR1; it is recorded here so that the research trajectory established in this phase remains open and traceable.

5.3.1 Problem Statement

MOSAIC is capable of serving results from two sources of substantially different freshness: exact SQL results computed over the Iceberg Gold tier, whose freshness is bounded by the CDC lag measured in Section 4.4 and is typically on the order of seconds to minutes, and approximate results computed over the pgvector index, whose freshness is bounded by the embedding-refresh schedule and is typically on the or-

der of hours. A representative query such as "find the ten products most relevant to user X" must combine an SQL-derived rank, based on purchase history, recency, and price range, with a semantically derived rank, based on embedding similarity. Reciprocal Rank Fusion, introduced in Section 3.5, is the natural baseline for this combination problem, but its formulation in Cormack et al. [7] ignores three properties specific to this setting: first, asymmetric staleness, since SQL data may be substantially fresher than the embeddings, so that a product that sold out five minutes ago can still receive a high rank from the semantic side while the SQL side already reflects the stock-out; second, heterogeneous score spaces, since an absolute SQL score and a normalized cosine-similarity score in the interval $[0,1]$ cannot be combined by a direct weighted sum without first being reconciled; and third, recall uncertainty, since an item absent from the approximate top-k result of the vector search is not necessarily irrelevant, but may simply have been missed by the approximate search procedure.

5.3.2 Research Question

The proposed research question is formulated as follows: in a unified data platform where structured analytics and vector search exhibit asymmetric freshness, how can a fusion function $f(\mathrm{SQL}_{\text{top-}k}, \mathrm{ANN}_{\text{top-}k}, \Delta t_{\mathrm{sql}}, \Delta t_{\mathrm{emb}})$ be designed to maximize the relevance and freshness of the combined result, relative to a freshness-unaware RRF baseline.

5.3.3 Proposed Framework

A staleness score is defined for each data source as the elapsed time since the item was last updated, normalized by an acceptable freshness window:

$$s_{\mathrm{sql}}(d) = \frac{\mathrm{now} - \mathrm{last_cdc}(d)}{T^{\mathrm{sla}}_{\mathrm{sql}}}, \qquad s_{\mathrm{emb}}(d) = \frac{\mathrm{now} - \mathrm{last_refresh}(d)}{T^{\mathrm{sla}}_{\mathrm{emb}}}. \tag{5.1}$$

On the basis of Equation (5.1), a freshness-weighted fusion score is proposed as

$$\mathrm{fusion}(d) = \alpha \cdot \mathrm{sql}(d) \cdot g\big(s_{\mathrm{sql}}(d)\big) + \beta \cdot \mathrm{ann}(d) \cdot g(s_{\mathrm{emb}}(d)), \tag{5.2}$$

where $g(\cdot)$ is a decay function, either linear or exponential, that reduces the confidence assigned to a source's score as that source's staleness increases. Equation (5.2) is deliberately left as a framework rather than a fully specified method, since determining the appropriate form of $g(\cdot)$ and the appropriate values of α and β is precisely the experimental work proposed for GR2.

5.3.4 Research Trajectory and Why the Problem Appears Novel

Table 5.1 outlines the proposed research trajectory across the remaining phases of the programme.

Table 5.1: Proposed research trajectory for the freshness-aware fusion direction.

Phase	**Content**	**Deliverable**
GR1	Survey of fusion methods; formalization of the problem statement and the proposed framework (this section)	Problem formalization
GR2	Implementation of staleness scoring from CDC-lag metadata; benchmark of freshness-weighted fusion against RRF under controlled staleness scenarios	Experimental comparison
Thesis	Learned fusion weights from implicit feedback; uncertainty bounds derived from the ANN recall guarantee; formal optimality conditions	Full research contribution

A review of the closest related work supports treating this problem as a genuine, if narrow, gap. RRF [7] is freshness-unaware by construction. Learned sparse-retrieval methods target relevance ranking within a single result set and do not address multi-source fusion. Temporal-ranking methods address the decay of relevance for an individual document over time, but not the fusion of two differently stale sources. Managed vector-search offerings from commercial lakehouse vendors currently expose vector search as an added feature without a unified freshness model spanning both the structured and the vector side. On this basis, the formalization of hybrid top-k fusion under asymmetric freshness, within the context of a single governed platform, is proposed as a candidate research direction; its genuine novelty, however, remains to be validated through the literature review planned as the first task of GR2.

This chapter has presented the three contributions considered most significant within GR1: the CDC comparison methodology, the two-track top-k taxonomy, and the proposed freshness-aware fusion problem. Chapter 6 concludes the report and sets out the planned direction for Graduation Research 2 and the Thesis.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

This report has presented MOSAIC, an open-source, self-hosted unified data platform designed to close three gaps identified in the current landscape of data-platform technologies: the absence of a benchmark-ready reference architecture for multi-source CDC, the lack of unified governance spanning structured analytics and vector search, and the absence of an optimized approach to hybrid top-k fusion under asymmetric data freshness. Relative to the managed-cloud, legacy-Hadoop, and do-it-yourself categories surveyed in Section 2.1, MOSAIC is positioned as a self-hosted platform that offers a documented reference architecture and an explicit benchmark design, at the cost of requiring in-house operational capability, which the deployable stack in Appendix B is intended to lower.

Within the scope of GR1, this research has produced a justified reference architecture spanning six layers, a data model for an e-commerce reference domain split across two heterogeneous sources, an explicit and honestly documented scope-decision record that separates the confirmed minimum viable core from every deferred component, a corrected two-track taxonomy for top-k optimization, and a formal problem statement for freshness-aware hybrid top-k fusion proposed as future work. What has not yet been produced, consistent with the feasibility-and-design framing established in Section 1.2, is an executed implementation or measured experimental results; these are the explicit objectives of Graduation Research 2. The principal lesson drawn from this design phase is that separating a broad, ambitious proposal into an honestly scoped minimum viable core and an explicitly deferred remainder, as recorded in Section **??**, produces a more defensible research plan than attempting to implement every originally proposed component within a single phase.

6.2 Future Work

The immediate future work is the implementation of the GR1 minimum viable core described in Section **??**, using the Docker Compose stack and connector configurations provided in Appendix B and Appendix C as a starting point, followed

by execution of the CDC-comparison and two-track top-k experiments designed in Section 4.4 and Section 4.5. Beyond completing the deferred MVP functions, three further directions are planned. First, the components deferred to GR2 in Table **??** – MongoDB as a third CDC source, Milvus as a comparison point for pgvector, and the Prometheus/Grafana observability stack – will be revisited once the core flow is validated. Second, the freshness-aware fusion direction formalized in Section 5.3 will be developed into a working implementation and benchmarked against the Reciprocal Rank Fusion baseline under controlled staleness scenarios. Third, for the Thesis phase, the fusion weights in Equation (5.2) are intended to be learned from implicit user feedback rather than fixed manually, and uncertainty bounds derived from the ANN recall guarantee are intended to be incorporated into the fusion score, moving the contribution from a formalized problem statement toward a validated, general solution.

BIBLIOGRAPHY

[1] M. Armbrust, A. Ghodsi, R. Xin, and M. Zaharia, "Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics," in *Proc. 11th Conf. Innovative Data Systems Research (CIDR)*, Chaminade, CA, USA, 2021.

[2] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, 2020.

[3] R. Fagin, A. Lotem, and M. Naor, "Optimal aggregation algorithms for middleware," in *Proc. 20th ACM SIGACT-SIGMOD-SIGART Symp. Principles of Database Systems (PODS)*, Santa Barbara, CA, USA, 2001, pp. 102–113.

[4] S. Agarwal, B. Mozafari, A. Panda, H. Milner, S. Madden, and I. Stoica, "BlinkDB: Queries with bounded errors and bounded response times on very large data," in *Proc. 8th ACM European Conf. Computer Systems (EuroSys)*, Prague, Czech Republic, 2013, pp. 29–42.

[5] Y. Park, B. Mozafari, J. Sorenson, and J. Wang, "VerdictDB: Universalizing approximate query processing," in *Proc. 2018 ACM SIGMOD Int. Conf. Management of Data*, Houston, TX, USA, 2018, pp. 1461–1476.

[6] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O'Reilly Media, 2017.

[7] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, Boston, MA, USA, 2009, pp. 758–759.

[8] Debezium Community, *Debezium Documentation*. [Online]. Available: `https://debezium.io/documentation/` (accessed Jul. 11, 2026).

[9] Apache Software Foundation, *Apache Iceberg Table Specification*. [Online]. Available: `https://iceberg.apache.org/spec/` (accessed Jul. 11, 2026).

[10] Trino Software Foundation, *Trino Documentation*. [Online]. Available: `https://trino.io/docs/current/` (accessed Jul. 11, 2026).

[11] Apache Software Foundation, *Apache Ranger Documentation*. [Online]. Available: `https://ranger.apache.org/` (accessed Jul. 11, 2026).

[12] DataHub Project, *DataHub Documentation*. [Online]. Available: `https://docs.datahub.com/` (accessed Jul. 11, 2026).

[13] pgvector Contributors, *pgvector: Open-source Vector Similarity Search for Postgres*. [Online]. Available: `https://github.com/pgvector/pgvector` (accessed Jul. 11, 2026).

[14] Amazon Web Services, "Optimizing read performance," *AWS Prescriptive Guidance – Apache Iceberg on AWS*. [Online]. Available: `https://docs.aws.amazon.com/prescriptive-guidance/latest/apache-iceberg-on-aws/best-practices-read.html` (accessed Jul. 11, 2026).

APPENDIX A

DATA MODEL AND ENTITY–RELATIONSHIP SCHEMA

This appendix lists the complete data definition language for the two relational sources introduced in Section 4.3, corresponding to the entity–relationship diagram in Figure 4.2.

A.1 MySQL 8.0 – Transactional Source

```
-- ============================================================================
-- MOSAIC :: MySQL 8.0 source schema  (E-Commerce TRANSACTIONAL data)
-- Role: high-frequency OLTP writes; captured via binlog (Debezium MySQL
    connector).
-- ============================================================================
CREATE DATABASE IF NOT EXISTS ecommerce_tx
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE ecommerce_tx;

-- ---- orders ----
CREATE TABLE orders (
    order_id        BIGINT       NOT NULL AUTO_INCREMENT,
    user_id         BIGINT       NOT NULL,              -- FK -> postgres
    users.id
    status
    ENUM('pending','paid','shipped','delivered','cancelled')
                                 NOT NULL DEFAULT 'pending',
    total_amount    DECIMAL(12,2) NOT NULL,
    currency        CHAR(3)      NOT NULL DEFAULT 'VND',
    region          VARCHAR(32)  NOT NULL,              -- used for bucket
    partitioning
    buyer_email     VARCHAR(255) NULL,                  -- PII: masked at
    SMT layer
    buyer_phone     VARCHAR(32)  NULL,                  -- PII: masked at
    SMT layer
    created_at      DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at      DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP
```

```
                                    ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (order_id),
    KEY idx_orders_user (user_id),
    KEY idx_orders_created (created_at)
) ENGINE=InnoDB;

-- ---- order_items ----
CREATE TABLE order_items (
    order_item_id   BIGINT       NOT NULL AUTO_INCREMENT,
    order_id        BIGINT       NOT NULL,
    product_id      BIGINT       NOT NULL,              -- FK -> postgres
    products.id
    quantity        INT          NOT NULL,
    unit_price      DECIMAL(12,2) NOT NULL,
    created_at      DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (order_item_id),
    KEY idx_items_order (order_id),
    KEY idx_items_product (product_id)
) ENGINE=InnoDB;

-- ---- payments ----
CREATE TABLE payments (
    payment_id      BIGINT       NOT NULL AUTO_INCREMENT,
    order_id        BIGINT       NOT NULL,
    method          ENUM('card','ewallet','cod','bank_transfer') NOT NULL,
    amount          DECIMAL(12,2) NOT NULL,
    raw_card_number VARCHAR(32)  NULL,                  -- PII: DROPPED at
    SMT layer
    status          ENUM('authorized','captured','failed','refunded') NOT
    NULL,
    created_at      DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (payment_id),
    KEY idx_pay_order (order_id)
) ENGINE=InnoDB;

-- ---- CDC replication user ----
-- Grants required by Debezium: SELECT (snapshot) + REPLICATION privileges.
CREATE USER IF NOT EXISTS 'cdc'@'%' IDENTIFIED BY 'cdcpw';
GRANT SELECT, RELOAD, SHOW DATABASES, REPLICATION SLAVE, REPLICATION CLIENT
    ON *.* TO 'cdc'@'%';
FLUSH PRIVILEGES;
```

A.2 PostgreSQL 15 – Master Data and Vector Store

```sql
-- ============================================================================
-- MOSAIC :: PostgreSQL 15 source schema  (E-Commerce MASTER data + VECTORS)
-- Dual role:
--   (1) CDC source (master data) captured via WAL / pgoutput logical decoding.
--   (2) Vector store for semantic search via the pgvector extension.
-- ============================================================================
CREATE EXTENSION IF NOT EXISTS vector;   -- pgvector

-- ---- categories ----
CREATE TABLE categories (
    id          BIGSERIAL PRIMARY KEY,
    name        TEXT      NOT NULL,
    parent_id   BIGINT    NULL REFERENCES categories(id),
    updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ---- users ----
CREATE TABLE users (
    id          BIGSERIAL PRIMARY KEY,
    full_name   TEXT      NOT NULL,
    email       TEXT      NULL,          -- PII: masked at SMT layer
    phone       TEXT      NULL,          -- PII: masked at SMT layer
    national_id TEXT      NULL,          -- PII: NEVER persisted (nulled at SMT layer)
    region      TEXT      NOT NULL,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ---- products ----
CREATE TABLE products (
    id           BIGSERIAL PRIMARY KEY,
    sku          TEXT      NOT NULL UNIQUE,
    name         TEXT      NOT NULL,
    description  TEXT      NULL,         -- source text for embeddings
    category_id  BIGINT    NOT NULL REFERENCES categories(id),
    price        NUMERIC(12,2) NOT NULL,
    updated_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

```
39
40 -- ---- inventory ----
41 CREATE TABLE inventory (
42     product_id   BIGINT    PRIMARY KEY REFERENCES products(id),
43     stock_qty    INT       NOT NULL DEFAULT 0,
44     warehouse    TEXT      NOT NULL,
45     updated_at   TIMESTAMPTZ NOT NULL DEFAULT now()
46 );
47
48 -- ---- product_embeddings  (pgvector; NOT a CDC source, populated by
       pipeline) -
49 -- 384-dim vectors from a sentence-transformers model (e.g.
       all-MiniLM-L6-v2).
50 CREATE TABLE product_embeddings (
51     product_id   BIGINT    PRIMARY KEY REFERENCES products(id),
52     embedding    vector(384) NOT NULL,
53     refreshed_at TIMESTAMPTZ NOT NULL DEFAULT now()   -- freshness signal
       (see Ch.10)
54 );
55
56 -- HNSW index for approximate top-k nearest-neighbour (cosine distance).
57 CREATE INDEX idx_prod_emb_hnsw
58     ON product_embeddings USING hnsw (embedding vector_cosine_ops)
59     WITH (m = 16, ef_construction = 64);
60
61 -- ---- CDC heartbeat sink (advances confirmed_flush_lsn; guards WAL
       bloat) -----
62 CREATE TABLE cdc_heartbeat (ts TIMESTAMPTZ NOT NULL);
63
64 -- ---- CDC replication user & logical publication ----
65 -- Debezium requires REPLICATION and a publication for pgoutput.
66 ALTER USER cdc WITH REPLICATION;
67 CREATE PUBLICATION mosaic_pub FOR TABLE users, products, categories,
       inventory;
68
69 -- ---- Example semantic top-k query (Track 2 of the two Top-k tracks) ----
70 -- SELECT p.id, p.name, 1 - (e.embedding <=> :q) AS similarity
71 -- FROM product_embeddings e JOIN products p ON p.id = e.product_id
72 -- ORDER BY e.embedding <=> :q      -- <=> = cosine distance operator
73 -- LIMIT 10;
```

APPENDIX B

DOCKER COMPOSE STACK

This appendix lists the single-node Docker Compose stack definition that deploys the complete GR1 minimum viable core described in Section **??**: MySQL and PostgreSQL sources, the Debezium/Kafka ingestion layer, Spark, Iceberg on MinIO with the Nessie catalog, Trino, Ranger, DataHub, and Airflow as the orchestration control plane.

```
# ============================================================================
# MOSAIC - Unified Lakehouse Platform (GR1 Proof-of-Concept stack)
# Single-node Docker Compose deployment for local feasibility study.
#
# Scope (GR1 MVP): MySQL + PostgreSQL CDC -> Kafka -> Spark -> Iceberg/MinIO
#                  -> Trino (federated SQL) ; pgvector for semantic search ;
#                  Ranger (RBAC/masking) + DataHub (lineage) ; Airflow control plane.
# Deferred (GR2/Thesis): MongoDB source, Milvus, Prometheus/Grafana.
# ============================================================================
version: "3.9"

x-common-env: &common-env
  TZ: Asia/Ho_Chi_Minh

networks:
  mosaic:
    driver: bridge

volumes:
  mysql-data:
  postgres-data:
  minio-data:
  kafka-data:

services:

  # ---- SOURCES ----
  mysql:
    image: mysql:8.0
    container_name: mosaic-mysql
    environment:
      <<: *common-env
      MYSQL_ROOT_PASSWORD: rootpw
      MYSQL_DATABASE: ecommerce_tx
      MYSQL_USER: cdc
      MYSQL_PASSWORD: cdcpw
    # binlog_format=ROW + row_image=FULL are mandatory for Debezium.
    command:
```

```
      - --server-id=1
      - --log-bin=mysql-bin
      - --binlog_format=ROW
      - --binlog_row_image=FULL
      - --expire_logs_days=3
      - --gtid_mode=ON
      - --enforce_gtid_consistency=ON
    volumes:
      - mysql-data:/var/lib/mysql
      - ./sql/mysql_schema.sql:/docker-entrypoint-initdb.d/01_schema.sql:ro
    ports: ["3306:3306"]
    networks: [mosaic]

  postgres:
    image: pgvector/pgvector:pg15   # PostgreSQL 15 with pgvector pre-installed
    container_name: mosaic-postgres
    environment:
      <<: *common-env
      POSTGRES_DB: ecommerce_md
      POSTGRES_USER: cdc
      POSTGRES_PASSWORD: cdcpw
    # wal_level=logical is mandatory for Debezium logical replication.
    command:
      - postgres
      - -c
      - wal_level=logical
      - -c
      - max_replication_slots=4
      - -c
      - max_wal_senders=4
      - -c
      - wal_keep_size=512MB       # mitigates WAL accumulation if Debezium lags
    volumes:
      - postgres-data:/var/lib/postgresql/data
      - ./sql/postgres_schema.sql:/docker-entrypoint-initdb.d/01_schema.sql:ro
    ports: ["5432:5432"]
    networks: [mosaic]

  # ---- INGESTION ----
  kafka:
    image: apache/kafka:3.7.0     # KRaft mode, no ZooKeeper
    container_name: mosaic-kafka
    environment:
      <<: *common-env
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
    volumes:
      - kafka-data:/var/lib/kafka/data
```

```
    ports: ["9092:9092"]
    networks: [mosaic]

  schema-registry:
    image: confluentinc/cp-schema-registry:7.6.0
    container_name: mosaic-schema-registry
    depends_on: [kafka]
    environment:
      <<: *common-env
      SCHEMA_REGISTRY_HOST_NAME: schema-registry
      SCHEMA_REGISTRY_KAFKASTORE_BOOTSTRAP_SERVERS: PLAINTEXT://kafka:9092
      SCHEMA_REGISTRY_LISTENERS: http://0.0.0.0:8081
    ports: ["8081:8081"]
    networks: [mosaic]

  # Debezium runs as connectors inside Kafka Connect. SMT masking happens here.
  kafka-connect:
    image: debezium/connect:2.7
    container_name: mosaic-connect
    depends_on: [kafka, schema-registry, mysql, postgres]
    environment:
      <<: *common-env
      BOOTSTRAP_SERVERS: kafka:9092
      GROUP_ID: mosaic-connect
      CONFIG_STORAGE_TOPIC: _connect_configs
      OFFSET_STORAGE_TOPIC: _connect_offsets   # Debezium source offsets (restart safety)
      STATUS_STORAGE_TOPIC: _connect_status
      KEY_CONVERTER: io.confluent.connect.avro.AvroConverter
      VALUE_CONVERTER: io.confluent.connect.avro.AvroConverter
      CONNECT_KEY_CONVERTER_SCHEMA_REGISTRY_URL: http://schema-registry:8081
      CONNECT_VALUE_CONVERTER_SCHEMA_REGISTRY_URL: http://schema-registry:8081
    ports: ["8083:8083"]
    networks: [mosaic]

  # ---- STORAGE ----
  minio:
    image: minio/minio:RELEASE.2024-06-13T22-53-53Z
    container_name: mosaic-minio
    environment:
      <<: *common-env
      MINIO_ROOT_USER: minioadmin
      MINIO_ROOT_PASSWORD: minioadmin
    command: server /data --console-address ":9001"
    volumes:
      - minio-data:/data
    ports: ["9000:9000", "9001:9001"]   # S3 API + console
    networks: [mosaic]

  # Iceberg REST catalog with git-like branching semantics.
  nessie:
    image: projectnessie/nessie:0.90.4
    container_name: mosaic-nessie
    environment:
      <<: *common-env
```

```
      NESSIE_VERSION_STORE_TYPE: IN_MEMORY   # swap to ROCKSDB/JDBC for persistence
    ports: ["19120:19120"]
    networks: [mosaic]

  # ---- PROCESSING ----
  spark:
    image: apache/spark:3.5.1
    container_name: mosaic-spark
    depends_on: [kafka, minio, nessie]
    environment:
      <<: *common-env
      AWS_ACCESS_KEY_ID: minioadmin
      AWS_SECRET_ACCESS_KEY: minioadmin
    # Spark Structured Streaming job reads Debezium topics -> writes Iceberg.
    command: ["sleep", "infinity"]   # jobs submitted via spark-submit in scripts/
    volumes:
      - ./spark:/opt/mosaic/spark:ro
    networks: [mosaic]

  # ---- ORCHESTRATION ----
  airflow:
    image: apache/airflow:2.9.2
    container_name: mosaic-airflow
    depends_on: [postgres]
    environment:
      <<: *common-env
      AIRFLOW__CORE__EXECUTOR: LocalExecutor
      AIRFLOW__DATABASE__SQL_ALCHEMY_CONN:
    postgresql+psycopg2://cdc:cdcpw@postgres/ecommerce_md
      # OpenLineage emission to DataHub without editing DAG code:
      AIRFLOW__OPENLINEAGE__TRANSPORT: '{"type":"http","url":"http://datahub-gms:8080"}'
    command: ["standalone"]
    volumes:
      - ./airflow/dags:/opt/airflow/dags:ro
    ports: ["8080:8080"]
    networks: [mosaic]

  # ---- SERVING ----
  trino:
    image: trinodb/trino:451
    container_name: mosaic-trino
    depends_on: [minio, nessie, mysql, postgres]
    volumes:
      - ./trino/catalog:/etc/trino/catalog:ro   # iceberg, mysql, postgresql catalogs
      - ./trino/ranger:/etc/trino/ranger:ro     # Ranger access-control plugin config
    ports: ["8090:8080"]
    networks: [mosaic]

  # ---- GOVERNANCE / LINEAGE ----
  ranger:
    image: apache/ranger:2.4.0
    container_name: mosaic-ranger
    depends_on: [postgres]
    environment:
```

```
      <<: *common-env
    ports: ["6080:6080"]   # Ranger admin UI (policies, column masking, row filters)
    networks: [mosaic]

  datahub-gms:
    image: acryldata/datahub-gms:v0.13.3
    container_name: mosaic-datahub
    depends_on: [kafka]
    environment:
      <<: *common-env
    ports: ["8092:8080"]   # DataHub GMS endpoint (receives OpenLineage events)
    networks: [mosaic]

# NOTE: For a full DataHub deployment (frontend, elasticsearch, mae/mce consumers)
# use the official datahub quickstart compose overlay. Kept minimal here for GR1.
```

APPENDIX C

DEBEZIUM CONNECTOR CONFIGURATIONS

This appendix lists the Debezium connector configurations referenced in Section 4.4, including the Kafka Connect Single Message Transforms that implement Layer-1 PII masking for each source.

C.1 MySQL Connector (Binlog and SMT Masking)

```
{
  "name": "mosaic-mysql-cdc",
  "config": {
    "connector.class": "io.debezium.connector.mysql.MySqlConnector",
    "tasks.max": "1",

    "database.hostname": "mysql",
    "database.port": "3306",
    "database.user": "cdc",
    "database.password": "cdcpw",
    "database.server.id": "184054",
    "topic.prefix": "mosaic.mysql",

    "database.include.list": "ecommerce_tx",
    "table.include.list":
     "ecommerce_tx.orders,ecommerce_tx.order_items,ecommerce_tx.payments",

    "snapshot.mode": "initial",

    "schema.history.internal.kafka.bootstrap.servers": "kafka:9092",
    "schema.history.internal.kafka.topic": "_schema_history.mosaic.mysql",

    "key.converter": "io.confluent.connect.avro.AvroConverter",
    "key.converter.schema.registry.url": "http://schema-registry:8081",
    "value.converter": "io.confluent.connect.avro.AvroConverter",
    "value.converter.schema.registry.url": "http://schema-registry:8081",

    "_comment_smt": "=== Layer-1 in-pipeline PII masking (irreversible,
     before Kafka) ===",
```

```
    "transforms": "unwrap,maskPII,dropSensitive,ts",

    "transforms.unwrap.type":
    "io.debezium.transforms.ExtractNewRecordState",
    "transforms.unwrap.drop.tombstones": "false",
    "transforms.unwrap.delete.handling.mode": "rewrite",
    "transforms.unwrap.add.fields":
    "op,source.ts_ms,source.file,source.pos",

    "transforms.maskPII.type":
    "org.apache.kafka.connect.transforms.MaskField$Value",
    "transforms.maskPII.fields": "buyer_email,buyer_phone",
    "transforms.maskPII.replacement": "",

    "transforms.dropSensitive.type":
    "org.apache.kafka.connect.transforms.ReplaceField$Value",
    "transforms.dropSensitive.exclude": "raw_card_number",

    "transforms.ts.type":
    "org.apache.kafka.connect.transforms.TimestampConverter$Value",
    "transforms.ts.field": "created_at",
    "transforms.ts.target.type": "string",
    "transforms.ts.format": "yyyy-MM-dd'T'HH:mm:ss'Z'"
  }
}
```

C.2 PostgreSQL Connector (WAL/pgoutput and SMT Masking)

```
{
  "name": "mosaic-postgres-cdc",
  "config": {
    "connector.class":
    "io.debezium.connector.postgresql.PostgresConnector",
    "tasks.max": "1",

    "database.hostname": "postgres",
    "database.port": "5432",
    "database.user": "cdc",
    "database.password": "cdcpw",
    "database.dbname": "ecommerce_md",
    "topic.prefix": "mosaic.postgres",

```

```
    "_comment_slot": "=== Logical replication slot + pgoutput plugin (WAL-based CDC) ===",
    "plugin.name": "pgoutput",
    "slot.name": "mosaic_slot",
    "publication.name": "mosaic_pub",
    "publication.autocreate.mode": "filtered",

    "schema.include.list": "public",
    "table.include.list": "public.users,public.products,public.categories,public.inventory",

    "snapshot.mode": "initial",

    "_comment_wal": "=== WAL accumulation guard: heartbeat advances confirmed_flush_lsn ===",
    "heartbeat.interval.ms": "10000",
    "heartbeat.action.query": "INSERT INTO public.cdc_heartbeat(ts) VALUES (now())",

    "key.converter": "io.confluent.connect.avro.AvroConverter",
    "key.converter.schema.registry.url": "http://schema-registry:8081",
    "value.converter": "io.confluent.connect.avro.AvroConverter",
    "value.converter.schema.registry.url": "http://schema-registry:8081",

    "_comment_smt": "=== Layer-1 in-pipeline PII masking (irreversible, before Kafka) ===",
    "transforms": "unwrap,maskPII,ts",

    "transforms.unwrap.type": "io.debezium.transforms.ExtractNewRecordState",
    "transforms.unwrap.drop.tombstones": "false",
    "transforms.unwrap.delete.handling.mode": "rewrite",
    "transforms.unwrap.add.fields": "op,source.ts_ms,source.lsn,source.txId",

    "transforms.maskPII.type": "org.apache.kafka.connect.transforms.MaskField$Value",
    "transforms.maskPII.fields": "email,phone,national_id",
    "transforms.maskPII.replacement": "",

    "transforms.ts.type": "org.apache.kafka.connect.transforms.TimestampConverter$Value",
    "transforms.ts.field": "updated_at",
```

```
    "transforms.ts.target.type": "string",
    "transforms.ts.format": "yyyy-MM-dd'T'HH:mm:ss'Z'"
  }
}
```